

A Note on the Post-Quantum Security of the Inverse Discrete Logarithm Problem

Joshua Limbrey and Andrew Mendelsohn

Department of EEE, Imperial College London, United Kingdom.
j.limbrey24@imperial.ac.uk, andrew.mendelsohn18@imperial.ac.uk

Abstract. In Submission 2025/1391 to the IACR Cryptology ePrint Archive, the *Inverse Discrete Logarithm Problem* (IDLP) is introduced and used to build a key exchange protocol and a KEM. The author claims both classical and post-quantum security for IDLP and therefore for the proposed protocols. It is the purpose of this note to give an efficient quantum algorithm for IDLP, based on the algorithm of Shor. We give an implementation of our algorithm, replacing the use of Shor’s algorithm with an oracle.

Keywords: post-quantum, cryptanalysis, discrete logarithm, Shor’s algorithm

1 Introduction

Since [3] proposed public-key cryptography from the Discrete Logarithm Problem (DLP), DLP has been central to modern cryptographic constructions. Unfortunately, Shor’s algorithm [6] renders DLP-based schemes insecure in a world of powerful quantum computers.

One response to the appearance of efficient quantum algorithms for discrete logarithms is to attempt to ‘tweak’ group-based discrete logarithm problems to render them infeasible to solve with said quantum algorithms. A recent example of this was SPDH [2], which attempted to use discrete logarithm-style problems in semidirect product groups to construct post-quantum secure protocols; this problem was subsequently cryptanalysed (and efficiently solved) in the sequence of works [5, 4, 1].

In this note we show that a recent attempt to revive this approach does not succeed. In particular, [7] defines the so-called ‘Inverse Discrete Logarithm Problem’ (IDLP), for which it claims post-quantum security, and from which it builds a key exchange and a key encapsulation mechanism. We show that Shor’s algorithm enables an adversary to solve IDLP efficiently and thus break the protocols based on IDLP.

2 The Inverse Discrete Logarithm Problem

We begin with the formal definition of IDLP:

Definition 1. Let: $m \in \mathbb{N}$ be either a prime or a product of two primes. Let $\varphi(\cdot)$ denote Euler's totient function, let $g \in (\mathbb{Z}/m\mathbb{Z})^*$ be a generator of $(\mathbb{Z}/m\mathbb{Z})^*$ sampled uniformly at random, and let $y \in \mathbb{Z}/m\mathbb{Z}$ satisfy $\gcd(y, \varphi(m)) = d > 1$. Define

$$\text{Mask}_{g,y}(x) := g^{xy} \bmod m$$

Then the Inverse Discrete Logarithm Problem (IDL) is, given (m, g, y, c) where $c = \text{Mask}_{g,y}(x)$ for some $x \in \mathbb{Z}/m\mathbb{Z}$, to recover any $\tilde{x} \in \mathbb{Z}/m\mathbb{Z}$ such that $c = g^{\tilde{x}y} \bmod m$.

We now provide the reasoning behind the claim that solving this problem is intractable, both classically and quantumly.

The scheme is suggested to be classically secure because there is a reduction from the (standard) discrete logarithm problem to IDL, which we sketch: suppose we have an IDL oracle and $g^x \bmod m$ is known for some public generator g . One then raises g^x to the power of d to obtain $g^{dx} \bmod m$, inputs (g, g^{dx}) to the IDL oracle to find $dx \bmod \varphi(m)$, and uses the information to gain non-negligible advantage in solving the DLP instance.

Regarding post-quantum security, the argument of [7] is that if one naively applies Shor's algorithm to (g, g^{xy}) , one recovers $xy \bmod \varphi(m) = xdy' \bmod \varphi(m)$ for some y' . Since $\gcd(d, \varphi(m)) > 1$, g^{xy} lies in one of d equivalence classes modulo exponentiation by d ; there are thus $\varphi(m)/d$ candidates for a solution x , which is prohibitively large to exhaustively search for cryptographic parameters.

The IDL problem can be used to build cryptographic functionality, as follows. We first describe the key exchange protocol, named Inverse Discrete Diffie–Hellman (IDDH): let (m, g, y) be a tuple of public parameters shared by Alice and Bob. Alice and Bob each sample a secret x_A, x_B respectively. Alice sends Bob $A = g^{x_A y} \bmod m$ and Bob sends Alice $B = g^{x_B y} \bmod m$. They each then compute

$$S = g^{x_A x_B y} \bmod m$$

We also note that [7] describes a KEM based on IDL. We omit the details of this scheme in this note, other than pointing out that the public key in the key encapsulation stage is of the form $\text{Mask}_{g,y}(x)$ for some public g, y .

We conclude this section by noting that [7] proposes parameters for implementations of IDL-based schemes; we give these parameters in 3.3.

3 Results

3.1 Using Shor's Algorithm to Solve IDL

Here we give our main result. Below, we assume our IDL instances are parameterised by a security parameter λ .

Theorem 1. *Suppose $d = \text{poly}(\lambda)$. Let $m = p$ or $m = pq$ for distinct odd primes p and q , and $1 < d < m$ be an integer satisfying $\gcd(d, m) = 1$. Let $y \in \mathbb{Z}$ satisfy $y = dy'$ with $\gcd(\varphi(m), y') = 1$. Then there is a polynomial-time quantum algorithm that, given IDLP inputs*

$$(m, g, y, c) = (m, g, y, \text{Mask}_{g,y}(x)) \in \mathbb{Z} \times (\mathbb{Z}/m\mathbb{Z})^* \times (\mathbb{Z}/m\mathbb{Z}) \times (\mathbb{Z}/m\mathbb{Z})^*$$

for some $x \in \mathbb{Z}/m\mathbb{Z}$, returns an $\tilde{x}$ such that $g^{\tilde{x}y} \bmod m = c$.

Proof. Begin by computing

$$y' := dy \bmod \varphi(m) \quad \text{and} \quad g' := g^{y'} \bmod m$$

Then submit $(g', \text{Mask}_{g,y}(x))$ as input to Shor's algorithm, which returns an x' such that $g'^{x'} = \text{Mask}_{g,y}(x)$ in quantum polynomial time, if such an x' exists. Note that $g^{y'}$ is a generator, as required. We have

$$g'^{x'} = \text{Mask}_{g,y}(x) \iff g^{x'y'} = g^{xy} = g^{x dy'}$$

if and only if $x' = xd \bmod \varphi(m)$.

It now suffices to find $xd \bmod m$ from $xd \bmod \varphi(m)$. This is efficiently computable because m is prime or semi-prime: if $m = p$ for some prime p then $\varphi(m) = p - 1$. Since we have $xd = x' + k(p - 1)$ for some non-negative integer $k \in \{0, 1, 2\}$, we find

$$xd = x' + k(p - 1) = x' - k + kp$$

so $xd \equiv x' - k \bmod m$ for some $k \in \{0, 1, \dots, d\}$. We may then plug in each value of k , multiply by d^{-1} (recall $\gcd(m, d) = 1$), recover $d + 1 = \text{poly}(\lambda)$ candidates for $x \bmod m$ and using the input data (m, g, y, c) check which value is correct.

Similarly, if $m = pq$ for distinct odd primes p, q , we have

$$xd = x' + k(p - 1)(q - 1)$$

for some non-negative integer $k \in \{0, 1, \dots, d\}$, and we find

$$xd = x' + k(p - 1)(q - 1) = x' + k(1 - p - q) + kpq$$

so $xd \equiv x' + k(1 - p - q) \bmod m$ for some $k \in \{0, 1, \dots, d\}$. Note we can assume p, q are public since we may factor m with Shor's algorithm. We may then plug in each value of k , multiply by d^{-1} , recover $d + 1$ candidates for $x \bmod m$ and using the input data (m, g, y, c) check which value is correct. $\square$

Note we may assume d is public because computing the gcd of two integers is classically easy.

3.2 Consequences for IDLP-based Protocols

We detail how our attack breaks the IDDH scheme described above. Alice and Bob send each other messages of the form $A = g^{x_A y} \bmod m$ and $B = g^{x_B y} \bmod m$ respectively. Thus an adversary who intercepts these messages may apply our algorithm to A and B to recover $\tilde{x}_A$ and $\tilde{x}_B \in \mathbb{Z}/m\mathbb{Z}$, thus compromising the shared key obtained at the end of the protocol.

A similar attack may be conducted on the KEM.

3.3 Implementation

We provide an implementation of the attack using the concrete parameters suggested in [7], given below:

```

m = 7875195248299573808823712291978888274442999959677694753912...
    5718477800872140027228586836890695257730117322116118363270137...
    06360435849189331163147852550661829
φ(m) = 7875195248299573808823712291978888274442999959677694753912...
    5718477800872140025453560375349892724590122210130854363603163...
    70101093947233788503313425120303276
y = 62992299088338401186198296729457514117356226056562890255129...
    14596518177159383400059722142269040719786180255780735904363854...
    210629937073277010069155086020014
g = 305288001113331764456583233448592165898869100805323680584266...
    95521042845849716583293356913629414008817122831212017030514517...
    6435676513862646086102301423957

```

For these parameters $d = 2$, which we will fix for the rest of the section.

We make use of an oracle in place of Shor's algorithm, which is established for each instance generated by `keygen`, returning x' such that $(g^{y'})^{x'} = g^{xy}$, or equivalently, $x' = 2x \bmod \varphi(m)$.

From here, there are two possible cases. In the first m is prime, in which case $\varphi(m) = m - 1$, and in the other m is semi-prime, so $\varphi(m) = (p - 1)(q - 1)$ where $m = pq$ with p, q distinct odd primes.

We find

$$x' = 2x \bmod \varphi(m)$$

Then, for $k \in \{0, 1, 2\}$:

<u>If m prime:</u>	<u>If m semi-prime:</u>
$2x = x' - k(m - 1)$	$2x = x' + k(p - 1)(q - 1)$
$\Rightarrow 2x = x' - k \bmod m$	$\Rightarrow 2x = x' + k(\varphi(m) - m) \bmod m$

There is now the matter of recovering x from the above, and in our case of $d = 2$, the possible values that k can take is sufficiently small to allow an exhaustive search to be conducted efficiently.

In both cases for m , we have that $x < m \Rightarrow 2x < 2m$, and so $k \in \{0, 1, 2\}$, allowing us simply to test whether each candidate $\tilde{x}$ satisfies $g^{\tilde{x}y} = c$, and returning once one is found.

We note that not always do we recover the original x , but we do always recover some $\tilde{x}$ that satisfies the challenge.

The full implementation that attacks the concrete parameter set can be found at https://github.com/supinie/IDDH_break. The parameter set includes $\varphi(m)$, which is not given in the formal definition of the problem; however, were it not given, it can be quantumly computed, again using Shor's algorithm¹. Since we are given it, we have no need to run Shor's algorithm to obtain it. The implementation imports the key generation algorithm from the implementation of IDDH found at <https://github.com/AdamaSoftware/InverseDiscrete>. Note that in the implementations provided accompanying the paper [7] at the given link, the key generation function used in all implemented schemes is the same, and so the same method we give to solve IDLP can be used to recover the key of any of the given schemes.

Algorithm 1 Break IDLP

Given $(m, g, y, g^{xy} \bmod m)$

```

1:  $2y' \leftarrow y$ 
2:  $base \leftarrow g^{y'}$ 
3:  $x' \leftarrow \text{Shor}(base, g^{xy})$   $\triangleright x' = 2x \bmod \varphi(m)$ 
4: if  $\varphi(m) = m - 1$  then
5:   for  $k$  in  $\{0, 1, 2\}$  do
6:      $\tilde{x} \leftarrow (x' - k) \cdot 2^{-1} \bmod m$ 
7:     if  $g^{\tilde{x}y} = g^{xy} \bmod m$  then
8:       return  $\tilde{x}$ 
9:     end if
10:  end for
11: else  $\triangleright \varphi(m) = (p - 1)(q - 1)$ 
12:   for  $k$  in  $\{0, 1, 2\}$  do
13:      $\tilde{x} \leftarrow (x' + k(\varphi(m) - m)) \cdot 2^{-1} \bmod m$ 
14:     if  $g^{\tilde{x}y} = g^{xy} \bmod m$  then
15:       return  $\tilde{x}$ 
16:     end if
17:   end for
18: end if
19: return  $\perp$ 

```

¹ If m is semi-prime, then computing $\varphi(m)$ is equivalent to factorisation; in the case of m prime, we must first verify before being able to efficiently compute $\varphi(m)$.

References

- [1] C. Battarbee, G. Borin, J. Brough, R. Cartor, T. Hemmert, N. Heninger, D. Jao, D. Kahrobaei, L. Maddison, E. Persichetti, A. Robinson, D. Smith-Tone, and R. Steinwandt. “On the Semidirect Discrete Logarithm Problem in Finite Groups”. In: *ASIACRYPT 2024*. Ed. by K.-M. Chung and Y. Sasaki. Vol. 15491. LNCS. Springer Nature Singapore, 2025, pp. 330–357. DOI: [10.1007/978-981-96-0944-4_11](https://doi.org/10.1007/978-981-96-0944-4_11).
- [2] C. Battarbee, D. Kahrobaei, L. Perret, and S. F. Shahandashti. “SPDH-Sign: Towards Efficient, Post-quantum Group-Based Signatures”. In: *PQCrypto 2023*. Ed. by T. Johansson and D. Smith-Tone. Vol. 14154. LNCS. Springer Nature Switzerland, 2023, pp. 113–138. DOI: [10.1007/978-3-031-40003-2_5](https://doi.org/10.1007/978-3-031-40003-2_5).
- [3] W. Diffie and M. Hellman. “New directions in cryptography”. In: *IEEE Transactions on Information Theory* 22.6 (1976), pp. 644–654. DOI: [10.1109/TIT.1976.1055638](https://doi.org/10.1109/TIT.1976.1055638).
- [4] M. Imran and G. Ivanyos. “Efficient quantum algorithms for some instances of the semidirect discrete logarithm problem”. In: *Designs, Codes and Cryptography* 92 (May 2024), pp. 2825–2843. DOI: [10.1007/s10623-024-01416-8](https://doi.org/10.1007/s10623-024-01416-8).
- [5] A. Mendelsohn, E. Dable-Heath, and C. Ling. “A small serving of mash: (Quantum) algorithms for SPDH-Sign with small parameters”. In: *Journal of Mathematical Cryptology* 19.1 (2025). DOI: [10.1515/jmc-2024-0025](https://doi.org/10.1515/jmc-2024-0025).
- [6] P. W. Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 1994, pp. 124–134. DOI: [10.1109/SFCS.1994.365700](https://doi.org/10.1109/SFCS.1994.365700).
- [7] M. Suslov. *Inverse Discrete Logarithm - Post-Quantum take on a classical problem*. Cryptology ePrint Archive, Paper 2025/1391. 2025. URL: <https://eprint.iacr.org/2025/1391>.